

Лабораторная работа №15

Изучение механизмов управления памятью

Цель работы

- 1.1 Изучить механизмы управления памятью в C#;

2 Литература

2.1 Фленов, М. Е. Библия C#. 4 изд / М. Е. Фленов. – Санкт-Петербург: БХВПетербург, 2019. – 512 с. – URL: <https://ibooks.ru/bookshelf/366634/reading>. – Режим доступа: только для зарегистрированных пользователей. – Текст : электронный.

3 Подготовка к работе

- 3.1 Повторить теоретический материал (см. п.2).
- 3.2 Изучить описание лабораторной работы.

4 Основное оборудование

- 4.1 Персональный компьютер.

5 Задание

- 5.1 Исследование работы сборщика мусора

- 5.1.1 Создать класс и метод

```
static Test globalRef;

void CreateObject()
{
    Test t = new Test();
    globalRef = t;
}

class Test
{
    ~Test()
    {
        Console.WriteLine("Object finalized");
    }
}
```

- 5.1.2 В функции Main вызвать метод CreateObject()

- 5.1.3 Вызвать принудительную сборку мусора (GC.Collect())

- 5.1.4 Проверить к какому поколению относится объект по глобальной ссылке (GC.GetGeneration(...))

- 5.1.5 Закомментировать строку globalRef = t;

- 5.1.6 Проверить вызов метода, определить различия, объяснить разницу в поведении.

- 5.2 Исследование размещения переменных в памяти

5.2.1 Создать класс PointClass с двумя публичными **полями** X и Y

5.2.2 Создать структуру PointStruct с двумя публичными **полями** X и Y

5.2.3 В функции Main создать объекты класса и структуры, установить значения полям.

5.2.4 Создать функцию ModifyClass, передать ей в параметры созданный класс и внутри функции присвоить полю класса другое значение

5.2.5 Создать функцию ModifyStruct, передать ей в параметры созданную структуру и внутри функции присвоить полю структуры другое значение

5.2.6 В функции Main вызвать созданные функции и проверить, изменились ли значения полей в классе и структуре.

5.2.7 Определить, почему значение в структуре не изменилось, изменить код так, чтобы структура тоже менялась в результате работы функции.

5.3 Изучение Large Object Heap.

5.3.1 Создать несколько объектов массивов размером 70000, 90000 и 100000 байт

5.3.2 Определить к какому поколению относится каждый из созданных объектов.

5.4 Использование IDisposable

5.4.1 Создать класс FileLogger (реализующий IDisposable), использующий объект StreamWriter (определен в классе как поле), для записи текста в файл логов методом Log.

5.4.2 Реализовать в классе метод Dispose, в котором высвободить ресурсы

5.4.3 Проверить работу класса с использованием и без использования IDisposable

5.5 Вызов неуправляемого кода

5.5.1 Дан некоторый формат пакета данных

```
byte[] packet =  
{  
    S 0x01, 0x00, // ushort ID = 1  
    t 0x10, 0x27, 0x00, 0x00, // int timestamp = 10000  
    r 0x00, 0x00, 0x48, 0x42, // float temperature = 50.0  
    e 0x01, // byte status  
    a 0x00, 0x00, 0x00, 0x00 // int checksum  
};
```

5.5.2 Необходимо реализовать метод, расшифровывающий содержимое массива с использованием unsafe-кода (результат возвращать в виде структуры)

5.5.3 Получить указатель на пакет данных:

```
// блок fixed нужен, чтобы GC не  
// переместил адреса во время обработки  
fixed (byte* ptr = packet)  
{  
    ushort* idPtr = (ushort*)ptr;  
    ushort id = *idPtr;
```

```
// считывание остальных значений  
};
```

5.5.4 Используя указатели, получить данные из пакета и вернуть результат пользователю.

5.5.5 Реализовать проверку контрольной суммы (сумма первых 11 байт пакета должна быть равна checksum)

6 Порядок выполнения работы

6.1 Запустить MS Visual Studio и создать консольное приложение C#.

6.2 Выполнить все задания из п.5 в одном решении.

6.3 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Чем стек отличается от кучи?

8.2 Для чего используются поколения в GC?

8.3 Что такое LOH?

8.4 Как работать с неуправляемым (небезопасным кодом) в C#?

8.5 Зачем используется IDisposable? Чем он отличается от финализатора?